

# P04 Monthly Calendar

## Overview

In this assignment, we are going to implement an object-oriented version of the Event Manager system from P01, now called Monthly Calendar. This enhanced version follows OOP principles, improving structure, maintainability, and reusability.

The program consists of the following classes:

- **Event** (Instantiable Class): Represents an event with a description, scheduled day, start time, and completion status. The class **Event** implements the **Comparable** interface.
- **MonthCalendar** (Instantiable Class): Manages a collection of events for a specific month and year, supporting event addition, removal, and retrieval.
- **CalendarTester** (Utility Class): Provides testing methods to verify the correctness of the **MonthCalendar** and **Event** classes.

This programming assignment builds upon the foundation of P01 Event Manager while introducing encapsulation, object interactions, implementing an interface, and method overriding to create a modular and well-structured monthly calendar for a simple event management system.

## Learning Objectives

The goals of this assignment include:

- **Learn** how to create and use **instantiable classes**.
- **Practice** the use of the `java.util.ArrayList` class.
- **Understand** the utility of working with multiple instances of a single class.
- **Explain** how object-oriented design influences the organization of code compared to P01.
- **Communicate** error states effectively using exceptions and appropriate messages, and understand how to handle thrown exceptions properly.
- Develop **unit tests** to verify the functionality of methods which that may throw exceptions.

## Grading Rubric

|           |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                |
|-----------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 5 points  | <b>Pre-Assignment Quiz:</b> The P04 pre-assignment quiz is accessible through Canvas before having access to this specification by <b>9:59PM CT on Friday February 21</b> .                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                    |
| 15 points | <b>Immediate Automated Tests:</b> Upon every submission of your assignment to Gradescope, you will receive immediate feedback from automated grading tests about whether specific parts of your submission conform to this write-up specification. If these tests detect problems in your code, they will attempt to give you some feedback about the kind of defect that they noticed. Passing all immediate automated tests does NOT guarantee full credit for the assignment.                                                                                                                                                                                                                                                                                                                                                                                                                               |
| 20 points |  <b>Additional Automated Tests:</b><br>The results of these tests will <b>NOT be available immediately</b> after each submission. Instead, the feedback from the Additional Automated Tests will be published on Gradescope <b>only after the assignment deadline</b> , alongside your manual grading feedback. These tests are similar to the Immediate Automated Tests but may evaluate different aspects of your submission or test your code in ways not covered by the Immediate Tests. It is important to note that you will <b>NOT have the opportunity to make up lost points or fix any bugs detected by the additional automated tests for credit</b> after the assignment deadline. Please ensure that you thoroughly test your submission on your own before the deadline to avoid missing points on these tests. |
| 10 points | <b>Manual Grading:</b> Human graders will review the commenting, style, and organization of your final submission. They will be checking whether it conforms to the requirements of the <a href="#">CS300 Course Style Guide</a> . You will NOT be able to resubmit corrections for extra points, and should therefore carefully review the readability of your code with respect to the course style guide.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                   |
| 50 points | <b>MAXIMUM Total Score</b>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                     |

## Assignment Requirements and Notes

### Pair Programming and Use of External Libraries

- Pair programming is **NOT ALLOWED** for this assignment.
- You are only allowed to **import** or **use** the libraries listed below in their respective files only.

- **No imports or external libraries** are permitted in your submitted `Event` class, except for reference types that are directly accessible through the `java.lang` package.
- **The ONLY external libraries** you are permitted to use in your submitted `MonthCalendar.java` and `CalendarTester.java` files are:  

```
import java.time.DateTimeException;
import java.time.YearMonth;
import java.util.ArrayList;
import java.util.NoSuchElementException;
```
- You are allowed to import the `java.util.Arrays` in your `CalendarTester.java` file **ONLY**.

## CITATIONS of Sources and Help

- Any source code, including Javadoc-style comments provided in this specification, may be included verbatim in your program without attribution.
- **ALL** other sources **MUST** be explicitly cited in your program comments, in accordance with the [Appropriate Academic Conduct guidelines](#).

### Important Note on CITATIONS:

- **NEW:** To receive full credit, your program must **CITE at least one source** in the header and include a corresponding **CITE:** in the method(s) where it was used. We **NO LONGER ACCEPT** “n/a” or “NONE.”
- Any use of **ChatGPT** or **other large language models (LLMs)** including Google AI, must be cited, AND your submission **MUST** include a **full transcript** - in a `log.txt` file, screenshots, or a link to the chat history (which some LLMs provide).
- Explicitly stating “**no outside sources**” will result in **zero points**. You are required to cite **any help received**, including assistance from course staff during consulting hours.
- Even if you did not receive help from a TA, peer mentor, or instructor, you should still cite lectures, ZyBooks, or other online resources.
- Examples of how to properly **CITE** resources in your program are provided on the [Academic Conduct](#) page on Canvas.
- **Failure to cite sources or include your logs** is considered **academic misconduct** and will be handled accordingly.

## Important Note on Exception Handling:

- When implementing methods in this assignment, **only throw exceptions that are explicitly declared** in the details of the Javadoc method header using the throws annotation. **No additional exceptions** should be thrown beyond those specified.
- If a method is not expected to throw any exceptions, ensure that you properly handle potential errors within the method instead of allowing unlisted exceptions to propagate. This may involve using appropriate try-catch blocks or validating inputs before performing operations.
- If a method is expected to throw an exception without being caught, let the exception propagate.

## CS300 Assignment Requirements

This section is **VALID** for **ALL** the CS300 assignments

- If you need assistance, please check the list of our [Resources](#).
- You can use the `java.lang.StringBuilder` class to efficiently build a string before returning it. Alternatively, you can create an empty string ("" ) and use simple concatenation (+) to construct the desired string.
- Read carefully through the specification provided in this write-up. This assignment requires clear understanding of its instructions. Read **TWICE** the instructions and do not hesitate to ask for clarification on piazza if you find any ambiguity.
- You **MUST NOT** add any additional fields either instance or static, and any public methods either static or instance to your program, other than those defined in this write-up.
- You **CAN** define local variables (declared inside a method's body) that you may need to implement the methods defined in this program.
- You **MUST NOT** add any additional fields either instance or static to your program, and any public methods either static or instance to your program, other than those defined in this write-up.
- You **CAN** define **private** methods to help implement the different public methods defined in this program, if needed.
- Your assignment submission must conform to the [CS300 Course Style Guide](#). Please review **ALL** the commenting, naming, and style requirements.

- If starter code files to download are provided, be sure to remove the comments including the `TODO` tags from your last submission to gradescope.
- Avoid submitting code which does not compile. Make sure that ALL of your submitted files ALWAYS compile. A submission which contains compile errors won't pass any of the automated tests on gradescope.
- You can submit your **work in progress (incomplete work) multiple times** on gradescope. Be sure to include method stubs for the incomplete methods (this includes complete method signature and a default return statement if the method return type is not void). Your submission may include methods not implemented or with partial implementation or with a default return statement.
- Run your program locally before you submit to Gradescope. If it doesn't work on your computer, it will not work on Gradescope.
- You are responsible for maintaining secure back-ups of your progress as you work. The OneDrive and GoogleDrive accounts associated with your UW NetID are often convenient and secure places to store such backups. Aspiring students may try their hands at [version control](#).
- Be sure to submit your code (work in progress) even incomplete of this assignment on [Gradescope](#) both early and often. This will 1) give you time before the deadline to fix any defects that are detected by the tests, 2) provide you with an additional backup of your work, and 3) help track your progress through the implementation of the assignment.

## 1 Getting Started

1. [Create a new project](#) in Eclipse, and name it something like "P04 Montly Calendar".
    - a. Ensure this project uses Java 17. Select "JavaSE-17" under "Use an execution environment JRE" in the New Java Project dialog box.
    - b. Do **not** create a project-specific package; use the default package.
  2. Download one (3) Java source file from the assignment page on Canvas:
    - `CalendarTester.java`, template for the Utility Tester Class.
  3. Create two (2) Java source files (**classes**) within your project's src folder:
    - `Event.java`
    - `MonthCalendar.java`
- Only the `CalendarTester` class should contain a `main()` method.

## 2 Implement and Test the Event Class

In this section, you will implement the **Event** class, which models an individual scheduled event with a description, date, and start time. The class also supports marking events as complete and comparing them based on their start time.

The full **Event** class documentation is available at the following link: [Event Class Javadocs](#).

Before implementing any methods, **add stubs** for all methods, including the constructor, with default return statements where needed.

### 2.1 Events: Data Fields

The **Event** class includes the following **private instance** fields:

- **description**: A brief text description of the event (cannot be null or blank).
- **isComplete**: A boolean flag indicating whether the event is marked as complete.
- **startHour**: The start hour of the event in 24-hour format (0-23).
- **startMin**: The start minute of the event (0-59).
- **startTime**: A computed integer representing the event's start time in HHMM format (e.g., 1:30 PM → 1330).
- **day**: The scheduled day of the event (1-31).

### 2.2 Events: Constructor

The constructor initializes an **Event** object with a description, scheduled day, and start time. The constructor must be public. It enforces input validation based on the following rules:

- The **description** must not be null or blank.
- The **day** must be in the range **1-31**.
- The start hour **startHour** must be between **0-23** (24-hour format).
- The start minute **startMin** must be between **0-59**.

If any of these constraints are violated, the constructor should throw an **IllegalArgumentException** with a descriptive error message.

## 2.3 Events: Methods and Testers

The `Event` class includes various accessor methods, mutator methods, and comparison logic. Notably, it implements the `Comparable<Event>` interface, allowing events to be sorted based on their start time. See the official documentation for [Comparable](#) for reference.

Implement ALL the methods in the `Event` class and the corresponding tester methods according to the provided javadocs.

- Before implementing any method, add a **stub** for every required method with an appropriate return type.
- Follow proper formatting and Java coding conventions.
- Use meaningful test cases to ensure robustness.

### Provided Method: `Event.toString()`

We provide you in the following with an implementation of the `Event.toString()` method. You can use it verbatim in your implementation. Be sure to CITE the write-up of p04 as source of this provided code.

---

```
/**
 * Provided method - Returns a String representation of this Event
 *
 * @return a String representation of this event
 */
@Override
public String toString() {
    return this.description + " at " + this.startHour + ":" + this.startMin
        + (isComplete ? " completed on Day " + day : "");
}
```

---

After implementing the `Event` class, you must thoroughly test it by implementing the following tester methods in `CalendarTester`.

```
testEventConstructorAndGettersValidBehavior()
testEventConstructorThrowingExceptions()
testEventCompareTo()
testEventEquals()
```

## 3 Implement and Test the `MonthCalendar` Class

In this section, you will implement the `MonthCalendar` class, which represents a **monthly event manager** for a specific year and month. This class allows users to add, remove, and

manage events, as well as track completed events.

The full `MonthCalendar` class documentation is available at the following link: [MonthCalendar Class Javadocs](#).

Before starting, **add stubs** for all methods, including a default return statement where needed. We strongly recommend implementing **tester methods first** to follow a **test-driven approach**.

### 3.1 Explore the `java.time.YearMonth` Class

The `MonthCalendar` class makes use of the `java.time.YearMonth` class to manage year and month values efficiently. This class provides built-in methods for date-related calculations:

- `YearMonth.of(int year, int month)`: Creates a `YearMonth` instance for a given year and month.
- `YearMonth.lengthOfMonth()`: Returns the number of days in the specified month.
- `YearMonth.getMonth().name()`: Returns the full name of the month (e.g., "February").
- `YearMonth.getMonthValue()`: Returns the month as an integer (1-12).

These methods will be useful when implementing the constructor and date-related methods in the `MonthCalendar` class.

### 3.2 `MonthCalendar`: Data Fields

The `MonthCalendar` class maintains event data using the following **private instance** fields:

- `MONTH`: A `YearMonth` object representing the year and month.
- `daysCount`: Stores the total number of days in the month.
- `events`: An **array of lists** where:
  - Each index represents a **day of the month**.
  - Each element at a given index is an `ArrayList<Event>` storing scheduled (uncompleted) events for that day.
  - The size of the array corresponds to **the number of days in the month**.
  - The list for **day  $i+1$**  is stored at index `i` of the array `events` (zero-based indexing).
- `completedEvents`: Stores **all completed events** for this month.

### 3.3 MonthCalendar: Constructor

The constructor initializes the `MonthCalendar` object for a given year and month, ensuring proper initialization of all instance fields.

**Implementation details:**

- Use `YearMonth.of(year, month)` to create the `MONTH` object.
- Validate that the year and month are within valid ranges. If not, throw a `DateTimeException`.
- Use `lengthOfMonth()` to determine the number of days in the month.
- Allocate the `events` array with the correct length.
- Initialize each element in `events` as an empty `ArrayList<Event>`.
- Initialize `completedEvents` as an empty `ArrayList<Event>`.

### 3.4 MonthCalendar: Methods and Testers

The `MonthCalendar` class includes methods for managing events, retrieving event lists, and marking events as complete. Implement all the required methods and constructor of the class `MonthCalendar` in accordance with the details provided in these javadocs ([MonthCalendar](#)).

**Provided Methods: `getCompletedEventsAsString` and `toString`**

We provide you in the following with an implementation of the `getCompletedEventsAsString()` and `MonthCalendar.toString()` methods. You can use them verbatim in your implementation. Be sure to CITE the write-up of p04 as source of this provided code.

---

```
/**
 * Provided method -- Returns a String representation of all completed events
 *
 * @return a String containing all completed events with their completed days
 * on separate lines, and an empty string if the list of completed events is empty.
 */
public String getCompletedEventsAsString() {
    String retval = "";
    for (Event e : completedEvents) {
        retval += e.toString() + "\n";
    }
    return retval.strip();
}
```

```

/**
 * Provided method -- Returns a String representation of all uncompleted events.
 *
 * Events scheduled on the same day must be in the increasing order.
 *
 * @return a String representation of All the events stored in the events list,
 * and an empty string if the list of events is empty.
 */
@Override
public String toString() {
    String retval = "";
    for (int i = 0; i < events.length; i++) {
        if (!events[i].isEmpty()) {
            retval += "Events for Day " + (i + 1) + ":\n";
            for (Event e : events[i]) {
                retval += e.toString() + "\n";
            }
        }
    }
    return retval.strip();
}
}

```

---

Below is a list of test methods you must implement in `CalendarTester` to ensure correctness.

```

testSuccessfulAddEvent()
testUnsuccessfulAddEvents()
testCancelEventValid()
testCancelEventExceptions()

```

## Important Notes

- Since the `Event` class overrides the `Object.equals()` method, the following `ArrayList<Event>` methods rely on `Event.equals()` for comparison:
  - `contains(Event)`
  - `indexOf(Event)`
  - `remove(Event)`
- Be sure to account for this behavior when implementing methods that involve searching or removing events.

## 4 Assignment Submission

**Congratulations on finishing this CS300 assignment!** After verifying that your work is correct, and written clearly in a style that is consistent with the [CS300 Course Style Guide](#), you should submit your final work through [Gradescope](#). The only three (3) files that you must submit are

- `Event.java`
- `MonthCalendar.java`
- `CalendarTester.java`

Your score for this assignment will be based on your “**active**” submission made prior to the assignment due date of Due: **9:59PM CT Thursday February 27**. The second portion of your grade for this assignment will be determined by running that same submission against additional offline automated grading tests after the submission deadline.

©**Copyright:** This write-up is a copyright programming assignment. It belongs to UW-Madison. This document should not be shared publicly beyond the CS300 instructors, CS300 Teaching Assistants, and CS300 Spring 2025 fellow students. Students are NOT also allowed to share the source code of their CS300 projects on any public site including GitHub, Bitbucket, etc.